

Project Blueprint: Pokémon AI Master Bot

This document serves as the comprehensive architectural blueprint for cloning and improving upon the "Future Sight AI" Pokémon Showdown bot. By utilizing a modern Python-based technology stack, we will bypass the slow web-scraping constraints of older models and build a high-speed, mathematically rigorous predictive engine capable of achieving a top-tier global ranking.

Core Objective

Develop an AI agent for Pokémon Showdown that operates without standard Machine Learning (Neural Networks). Instead, it will use real-time probability mathematics, state-tree simulation, and inverse data calculations to analytically outplay human opponents in Generation 9 (or older) formats.

1. The Technology Stack (What We Need)

The original Future Sight AI struggled with performance due to running JavaScript multi-threading over a browser client. We will fix this by separating the server communication from the internal simulation engine.

- **Language:** Python 3.10+ (Primary logic, data handling, algorithm orchestration).
- **Network API (`poke-env`):** An async Python library designed specifically to interface with the Pokémon Showdown WebSocket API. This replaces Selenium completely.
- **Simulation Engine (`@pkmn/engine`):** A hyper-optimized engine written in Zig. We will use Python bindings to run this engine locally, allowing the AI to simulate thousands of board states per second.
- **Data Handling:** `requests` (to fetch Smogon JSON data) and `numpy` (for fast matrix calculations in the Expectiminimax algorithm).

2. Architectural Phases (How to Proceed)

Phase 1: The Server Bridge & Infrastructure

The first step is simply getting a bot onto the ladder that can see the game state and issue legal moves.

- **Task:** Instantiate a `Player` class using `poke-env`.
- **Task:** Configure asynchronous loops (`asyncio`) to handle server messages in real-time.

- **Goal:** The bot can log into Pokémon Showdown, accept challenges, parse the active battle state (health, active Pokémon, weather), and pick a random legal move without crashing.

Phase 2: The Data Layer (Solving Hidden Info)

Unlike Chess, Pokémon features hidden information (opponent's held items, EVs/IVs, unrevealed moves). A standard bot will be paralyzed by the near-infinite possibilities. We use "Priors" to solve this.

- **Task:** Build a scraper/fetcher targeting the Smogon Usage Statistics API.
- **Task:** Create a localized mapping database. When the opponent sends out a Pokémon (e.g., *Garchomp*), the bot instantly queries the database to load its 4 most statistically common competitive movesets, items, and abilities.
- **Goal:** Prune the decision tree. We only simulate futures based on what the opponent is *statistically likely* to do, discarding irrelevant noise.

Phase 3: Inverse Damage Calculation

This is the secret weapon of the AI. By reversing the Pokémon damage formula, the AI can mathematically confirm hidden variables deduced probabilistically in Phase 2.

$$\text{Damage} = (((2 \times \text{Level} / 5) + 2) \times \text{Power} \times A/D) / 50 + 2) \times M$$

- **Task:** Implement a listener that captures exact HP pixel loss when the bot takes a hit.
- **Task:** Isolate the **M** (Modifier) and **A/D** (Attack/Defense) variables. By cross-referencing the resulting multiplier against the bot's known defense, the AI can determine if the opponent is holding a *Choice Band*, has a boosting ability, or has maximum Attack effort values.
- **Goal:** Update the game state with absolute facts midway through a battle, sharpening the simulation accuracy.

Phase 4: The Simulation Engine & Algorithm

This is where the bot actually "thinks." We plug our known data and statistical assumptions into a custom search algorithm.

- **Task:** Integrate the Zig-based `pkmn/engine` into the Python script.
- **Task:** Write a **Simultaneous Expectiminimax** algorithm. Standard Minimax fails because both players move at the same time, and moves have random elements (accuracy, critical hits, damage rolls).

- **Logic Flow:**

1. Branch 1: Bot picks Move A, Opponent picks Move B.
2. Multiply the resulting board state value by the RNG probability (e.g., Move B has 80% accuracy, so calculate the "Hit" branch at 0.8 weight, and "Miss" branch at 0.2 weight).
3. Simulate 3 to 4 turns deep.
4. Return the move with the highest Expected Value (EV).

3. Step-by-Step Implementation Roadmap

Important: Do not attempt to build the search algorithm until the network bridge and data layers are 100% stable. Flawed game state data will cause the algorithm to hallucinate winning paths.

Follow this chronological order for development:

1. **Setup:** Initialize the Python environment, install `poke-env`, and create a test account on Pokémon Showdown.
2. **The Dummy Bot:** Write a basic script that accepts battles and clicks random moves just to test WebSocket stability.
3. **The Statistician:** Write the JSON parsers for the Smogon API. Verify that the bot can print out the opponent's most likely moves in the console during a live random battle.
4. **The Engine Integration:** Hook up the `pkmn/engine`. Pass the current `poke-env` game state into the Zig engine and verify it successfully simulates one turn into the future.
5. **The Math Core:** Implement the Inverse Damage Calculator functions. Run tests by battling the bot yourself and seeing if it correctly guesses your held item based on your damage output.
6. **The Brain:** Write the Expectiminimax loop. Tie the simulation engine and the Smogon prior data together to evaluate the tree.
7. **Optimization:** Use Python profiling (e.g., `cProfile`) to find bottlenecks in the search tree. Adjust the search depth based on the strict 150-second Showdown timer limits.

"To master a game, you need a plan, and a good place to start is to find how many different places you might have to use it."